RAI-818: INTELLIGENT MOBILE ROBOTICS

Sampling Based Methods

- **Probabilistic Roadmap Approaches**
  - **Phases**
  - **Applications**
  - **Post Processing**
  - **Connection Strategies**
  - **Narrow Passage Problem**

Text : Principles of Robot Motion – Theory, Algorithms and Implementations
H. Choset, K.M. Lynch, S. Hutchinson, G. Kantor, W. Burgard, et al.

INSTRUCTOR:
DR. KUNWAR FARAZ AHMED KHAN

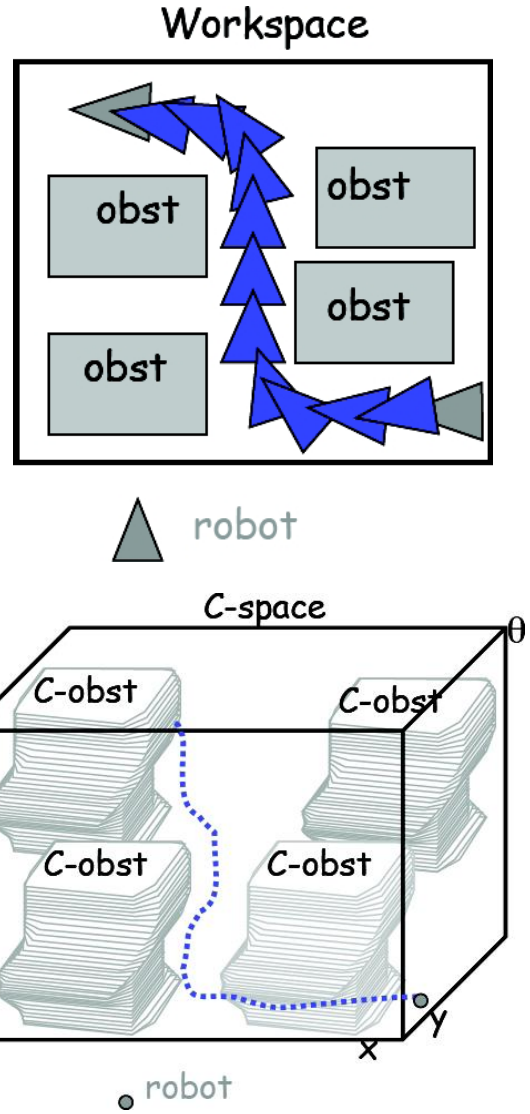
Sampling Based Methods

- Revision

Revision : Path Planning

- From Workspace to Configuration Space
  - simple workspace obstacle transformed into complex configuration-space obstacle
  - robot transformed into point in configuration space
  - path transformed from swept volume to 1d curve

- Explicit Construction of Configuration Space/Roadmaps
  - PSPACE-complete
  - Exponential dependency on dimension
  - Not practical algorithms



Defining futures

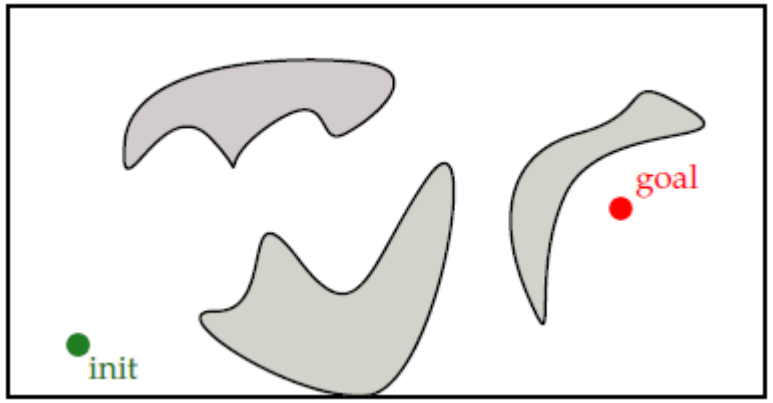
Sampling Based Methods

- Revision

Revision: Motion Planning for Point Robot

How would you solve it?

- *Robotic System*: Single point
- *Task*: Compute collision-free path from initial to goal position



- *Problem Areas:*
  - Hard to plan for many d.o.f robots
  - Computation complexity for high dimensional configuration spaces would grow exponentially
  - Potential fields run into local minima
  - Complete, general purpose algorithms are at best exponential and have not been implemented

Defining futures

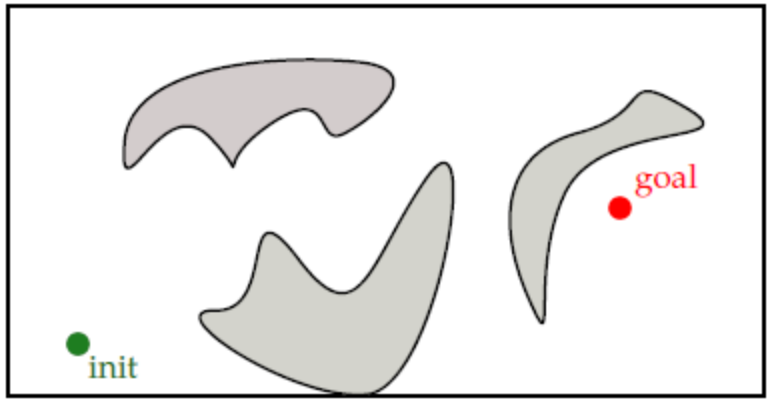
Sampling Based Methods

- Revision
- Sampling Methods

Sampling Methods: Motion Planning for Point Robot

How would you solve it?

- *Robotic System*: Single point
- *Task*: Compute collision-free path from initial to goal position



- *Monte-Carlo Idea:*
  - Define input space
  - Generate inputs at random by sampling the input space
  - Perform a deterministic computation using the input samples
  - Aggregate the partial results into final result

Defining futures

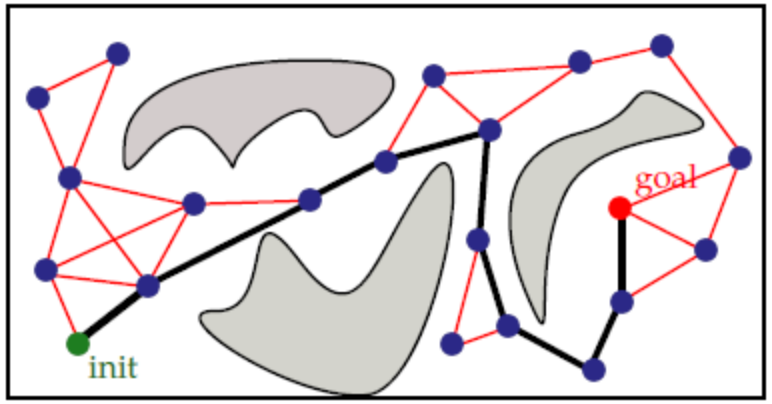
Sampling Based Methods

- Revision
- Sampling Methods

Revision : Motion Planning for a Point Robot

- *Robotic System*: Single point
- *Task*: Compute collision-free path from initial to goal position

How would you solve it?



- *Sample Points*
- *Discard points that are in collision*
- *Connect neighbouring samples via straight-line segments*
- *Discard straight-line segments that are in collision*

→ *Gives rise to a graph, called the roadmap*

→ *Collision-free path can be found by performing graph search*

Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- PRM Approach

Probabilistic Road Map (PRM) Method

0. Initialization

add q_{init} and q_{goal} to roadmap vertex set V

1. Sampling

repeat several times

$q \leftarrow Sample\ (\ )$

if $IsCollisionFree\ (q) = true$

add q to roadmap vertex set V

2. Connect Samples

for each pair of neighbouring samples $(q_a, q_b) \in V \times V$

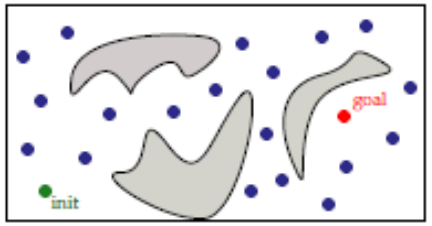
$path \leftarrow GenerateLocalPath\ (q_a, q_b)$

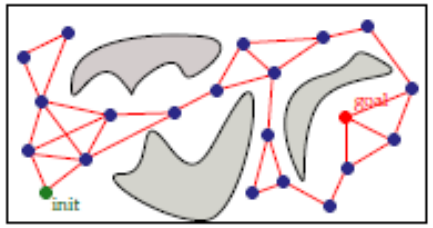
if $IsCollisionFree\ (path) = true$

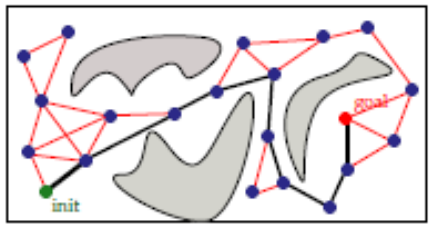
add (q_a, q_b) to roadmap edge set E

3. Graph Search

search graph (V, E) for path from q_{init} to q_{goal}







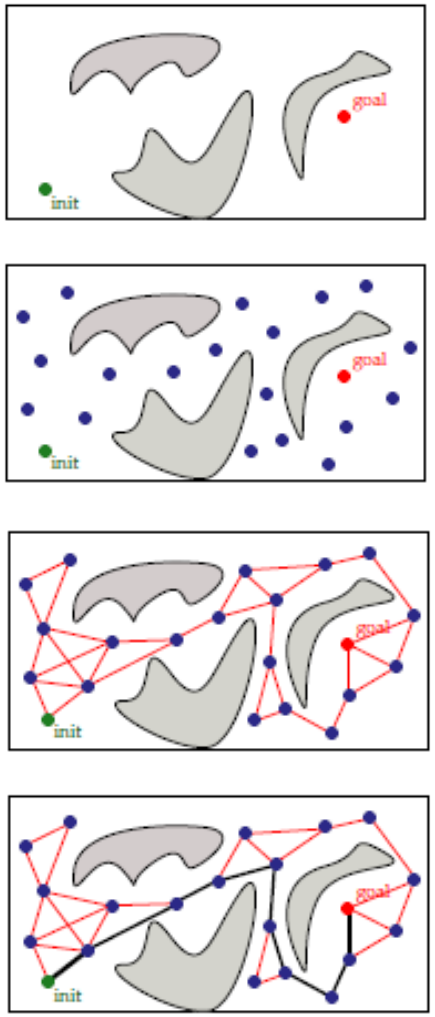
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- PRM Approach

Probabilistic Road Map (PRM) Method

- *Repeat n times*
  - *Generate a random point in configuration space, x*
  - *If x is in free space*
    - *Find the k closest points in the roadmap to x according to the Dist Function*
    - *Try to connect the new random sample to each of the k neighbors using the LocalPlanner procedure. Each successful connection forms a new edge in the graph.*

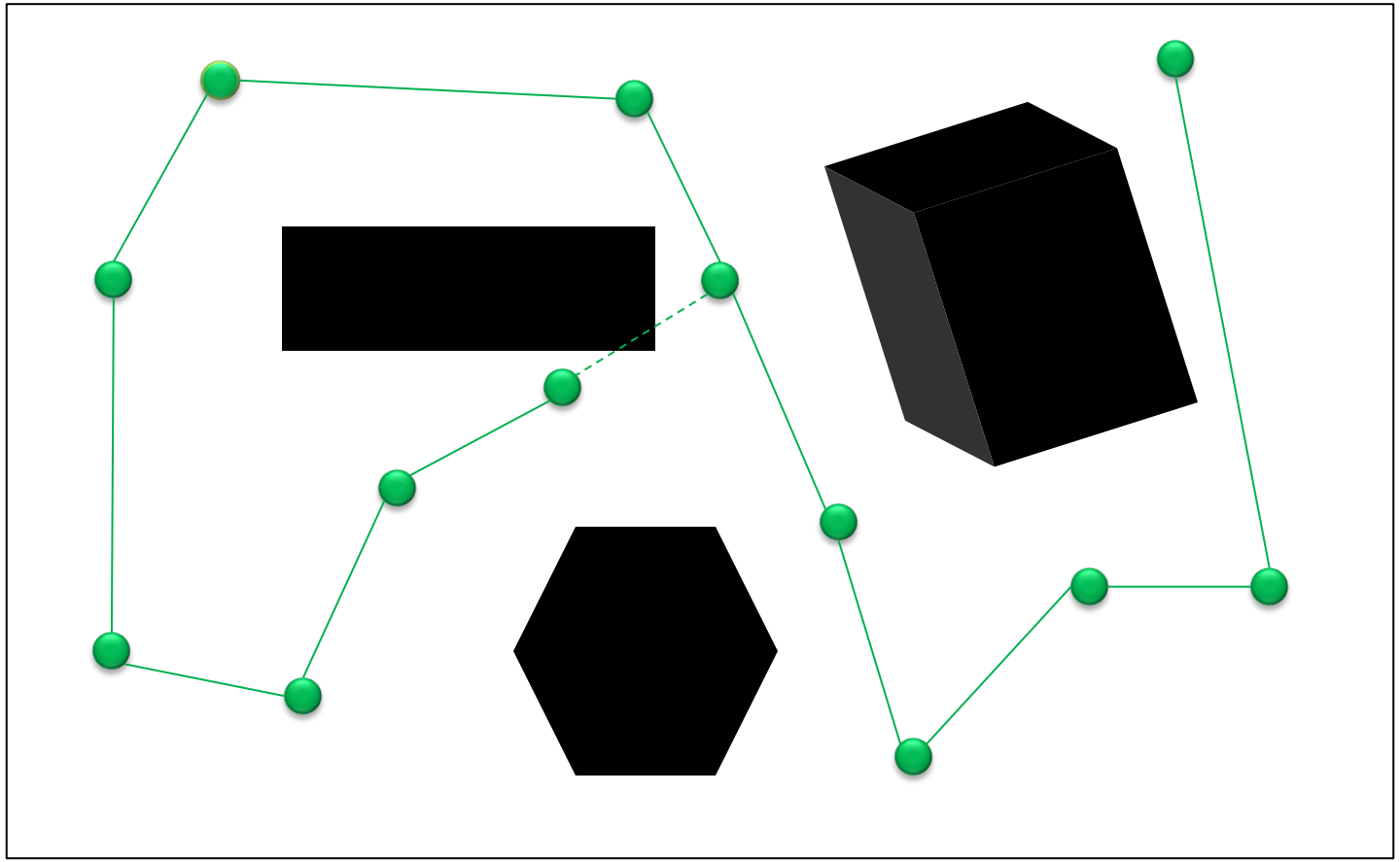


Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- PRM Approach

Probabilistic Road Map (PRM) Method



If the value of *k* set as 1?

Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- PRM Approach

Probabilistic Road Map (PRM) Method

Dist Function

- The PRM procedure relies upon a distance function, Dist that can be used to gauge the distance between two points in configuration space. This function takes as input the coordinates of the two points and returns a real number

$$Dist(\mathbf{x}, \mathbf{y}) \in \mathbb{R}$$

- Common choices for distance functions include

The L1 distance :

$$Dist_1 = \sum_i |\mathbf{x}_i - \mathbf{y}_i|$$

$$\boldsymbol{Dist} = |\boldsymbol{x_1} - \boldsymbol{x_2}| + |\boldsymbol{y_1} - \boldsymbol{y_2}|$$

The L2 distance :

$$Dist_2 = \sqrt{(\sum_i (\mathbf{x}_i - \mathbf{y}_i)^2)}$$

$$\boldsymbol{Dist} = \sqrt{(\boldsymbol{x_1} - \boldsymbol{x_2})^2 + (\boldsymbol{y_1} - \boldsymbol{y_2})^2}$$

Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**

Probabilistic Road Map (PRM) Method

- *Advantages*
  - Computationally efficient
  - Solves high-dimensional problems (with hundreds of DOFs)
  - Easy to implement
  - Applications in many different areas
- *Disadvantages*
  - Does not guarantee completeness
- Is it then just a heuristic approach? No. It's more than that
- *It offers probabilistic completeness*
  - When solution exists - probabilistically complete planner finds a solution with probability as time goes to infinity.
  - When a solution does not exists - probabilistically complete planner may not determine that a solution does not exist.

Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**

Phases of PRMs

- *Learning Phase*
  - Construction Step
  - Expansion Step
- *Query Phase*

Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - **Construction**

Phases of PRMs – Learning Phase

- Construction Step
  - Construct a probabilistic roadmap by generating random free configurations of the robot and connecting them using a simple, but very fast motion planner, also know as a *local planner*
  - Store as a graph whose nodes are the configurations and whose edges are the paths computed by the *local planner*

Sampling Based Methods

- Revision
- Sampling Methods
- PRM Approach
  - Phases
    - + Learning Phase
      - ▪ Construction

Phases of PRMs – Learning Phase

- **Construction Step**

Input: *geometry of the moving object & obstacles*

Output: roadmap $G = (V, E)$

1: $V \leftarrow \varnothing$ and $E \leftarrow \varnothing$.
2: repeat
3: $q \leftarrow$ a configuration sampled uniformly at random from C
4: if $CLEAR(q)$ then
5: Add q to V
6: $N_q \leftarrow$ a set of nodes in V that are close to q
6: for each $q' \in N_q$, in order of increasing $d(q, q')$
7: if $LINK(q, q')$ then
8: Add an edge between q and q' to E.

Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - **Construction**

Phases of PRMs – Learning Phase

- Construction Step
  - Initially, the graph $R = (N, E)$ is empty
  - Then, repeatedly, a *random free configuration* is generated and added to N
  - For every new node c, select a number of nodes from N and try to connect c to each of them using the *local planner*.
  - If a path is found between c and the selected node n, the edge (c, n) is added to E. The path itself is not memorized.

Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - **Construction**

Phases of PRMs – Learning Phase

- Construction Step – Determine a Random Free Configuration
  - We want nodes of *R* to be a rather uniform sampling of *C*
    - Draw each of its coordinates from the interval of values of the corresponding degrees of freedom. (Use the *uniform probability distribution* over the interval)
    - Check for *collision* both with robot itself and with obstacles
    - If *collision free*, add to *N*, otherwise *discard*

Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - **Construction**

Phases of PRMs – Learning Phase

- Construction Step – Local Path Planner
  - Can pick different ones
    - Nondeterministic – have to store local paths in roadmap
    - Powerful - slower but could take fewer nodes but takes more time
    - Fast - less powerful, needs more nodes
  - Go with the fastest one
    - Need to make sure start and goal configurations can connect to graph, which requires a somewhat dense roadmap
    - Can reuse local planner at query time to connect start and goal configurations
    - Don't need to memorize local paths

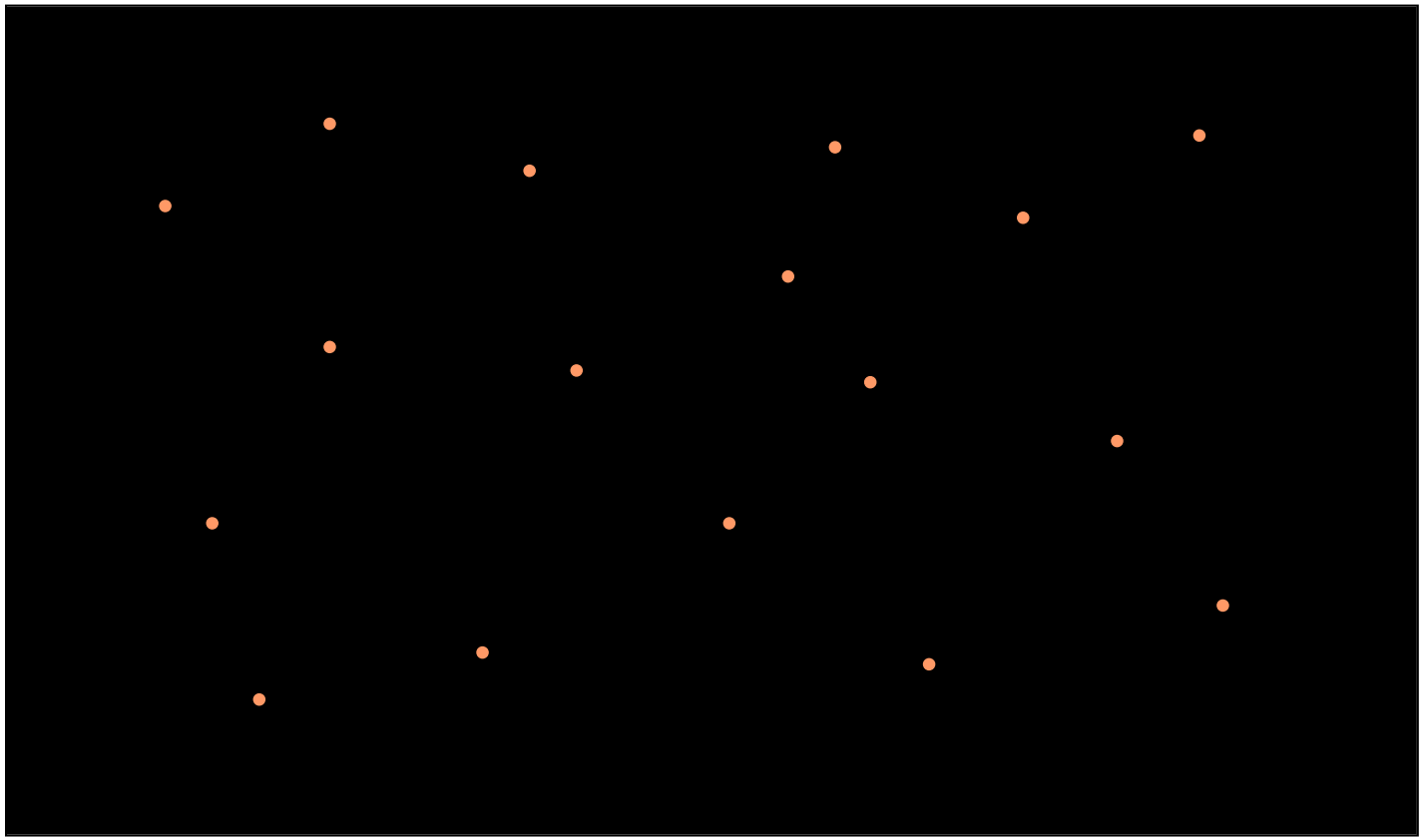
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - **Construction**

Phases of PRMs – Learning Phase

- Construction Step – Random Configurations



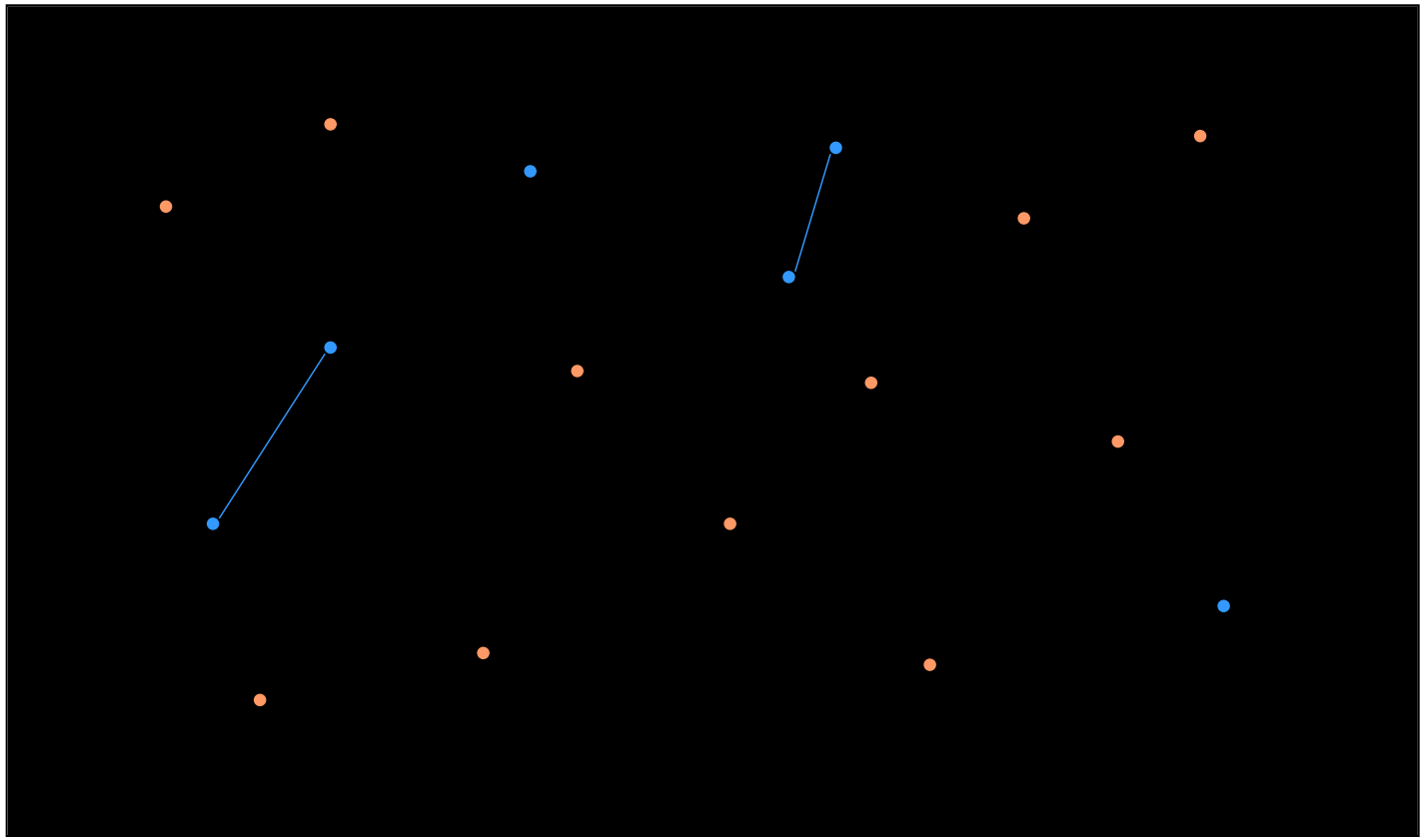
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - **Construction**

Phases of PRMs – Learning Phase

- Construction Step – Updating Neighboring Edges



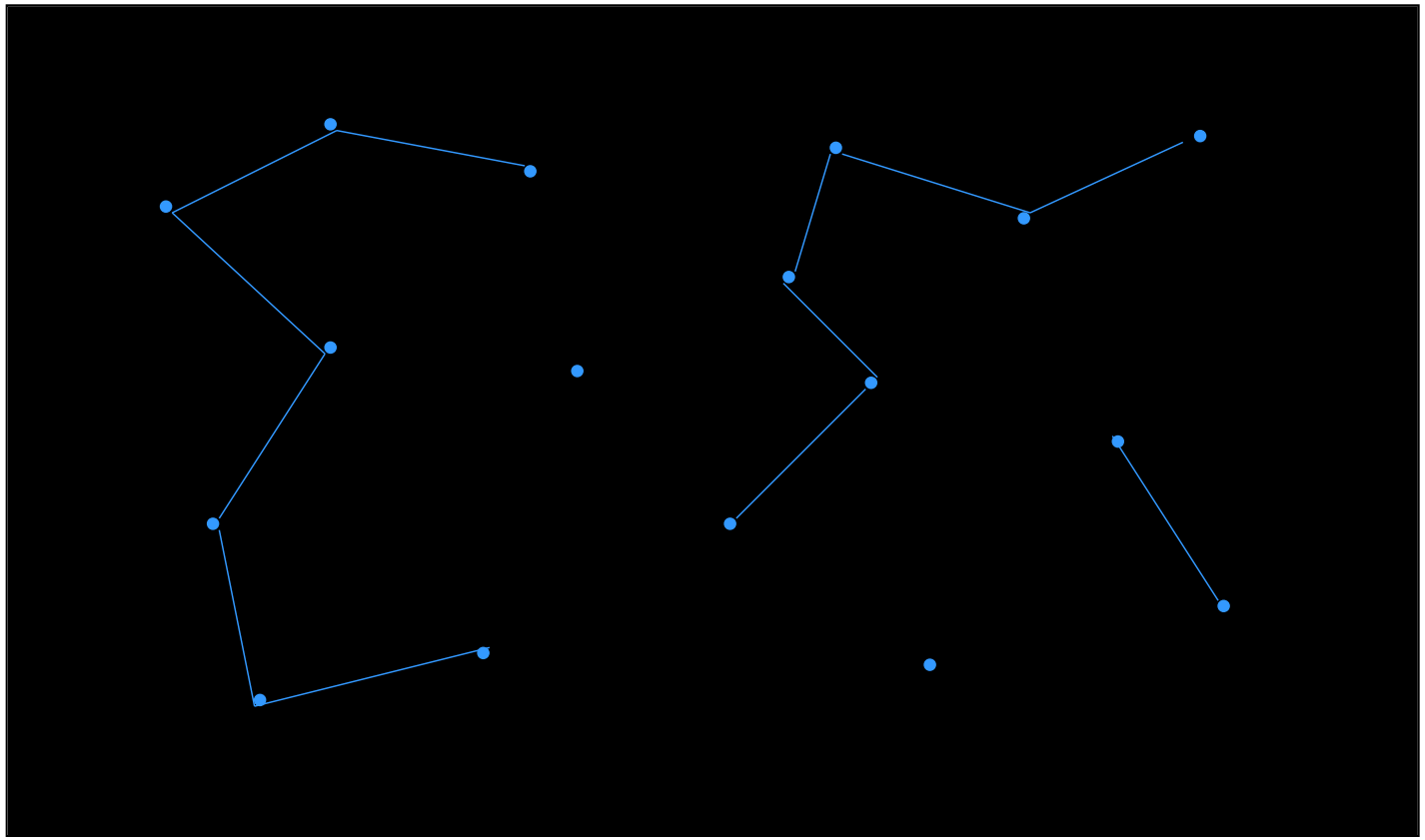
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - **Construction**

Phases of PRMs – Learning Phase

- Construction Step – Completed



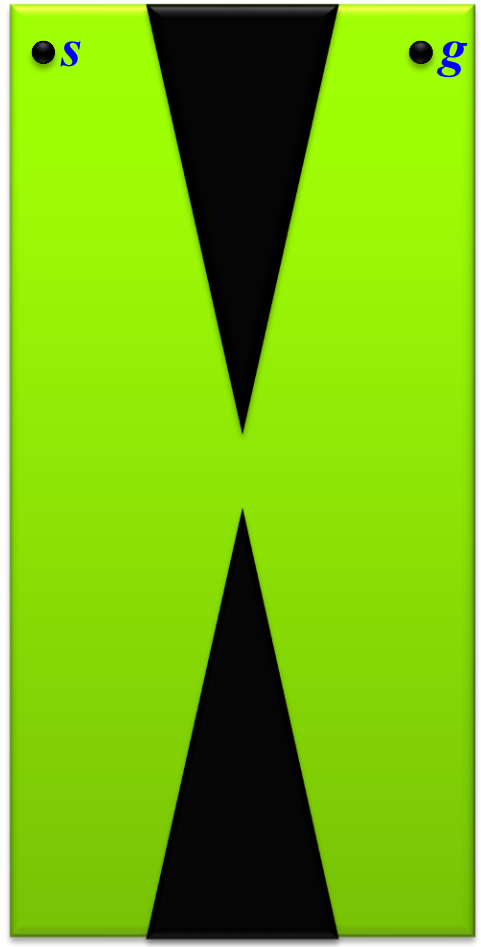
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - Construction
      - **Expansion**

Phases of PRMs – Learning Phase

- Expansion Step
  - Sometimes R consists of several large and small components which do not effectively capture the connectivity of C_{free}
  - The graph can be disconnected at some narrow region
  - Assign a positive weight $w(c)$ to each node c in N
  - $w(c)$ is a heuristic measure of the *difficulty* of the region around c. So, $w(c)$ is large when c is in a difficult region. We normalize w so that all weights together add up to one. The higher the weight, the higher the chances the node will get selected for expansion.



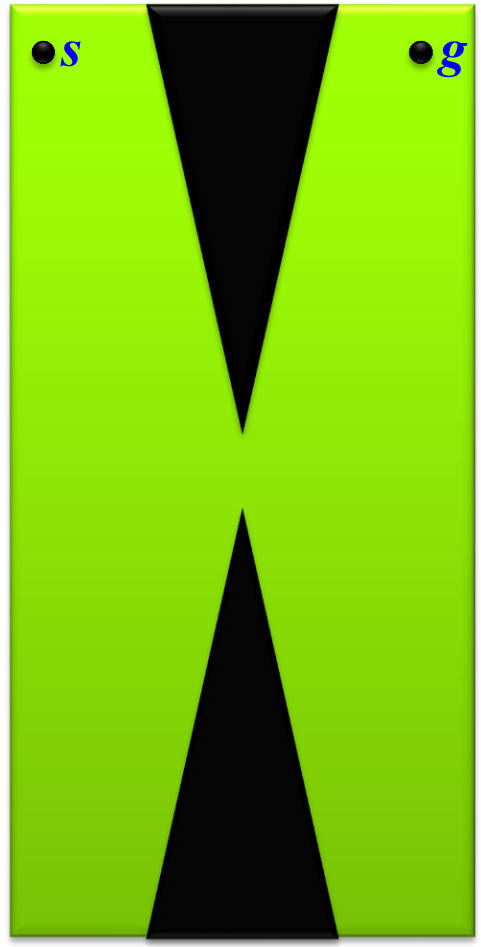
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - Construction
      - **Expansion**

Phases of PRMs – Learning Phase

- Expansion Step – How to choose w(c)
  - Can pick different heuristics
    - Count number of nodes of N lying within some predefined distance of c.
    - Check distance d_c from c to nearest connected component not containing c.
    - Use information collected by the local planner. (If the planner often fails to connect a node to others, then this indicates the node is in a difficult area).



Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - Construction
      - **Expansion**

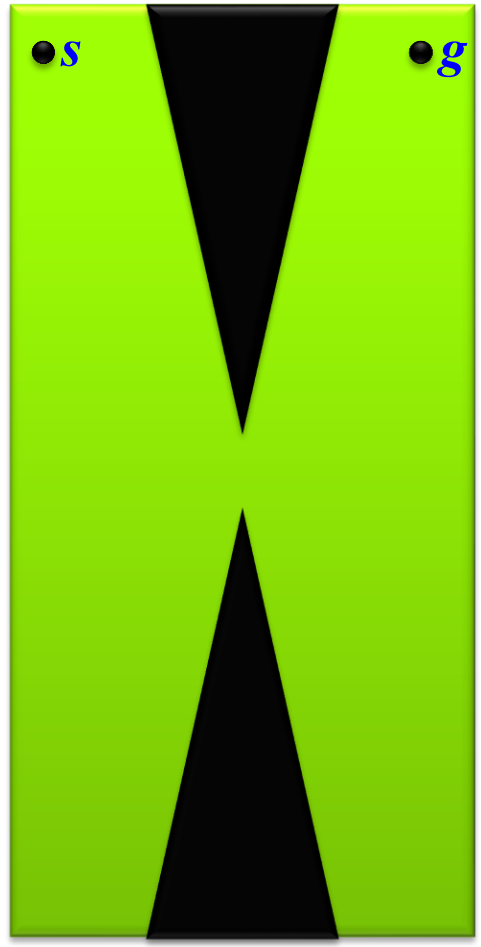
Phases of PRMs – Learning Phase

- Expansion Step – How to choose w(c)
  - One Method

    At the end of the construction step, for each node c, compute the failure ratio $r_f(c)$ defined by:

$$r_f(c) = \frac{f(c)}{n(c)+1}$$

    where $n(c)$ is the total number of times the local planner tried to connect c to another node and $f(c)$ is the number of times it failed.



Defining futures

Sampling Based Methods

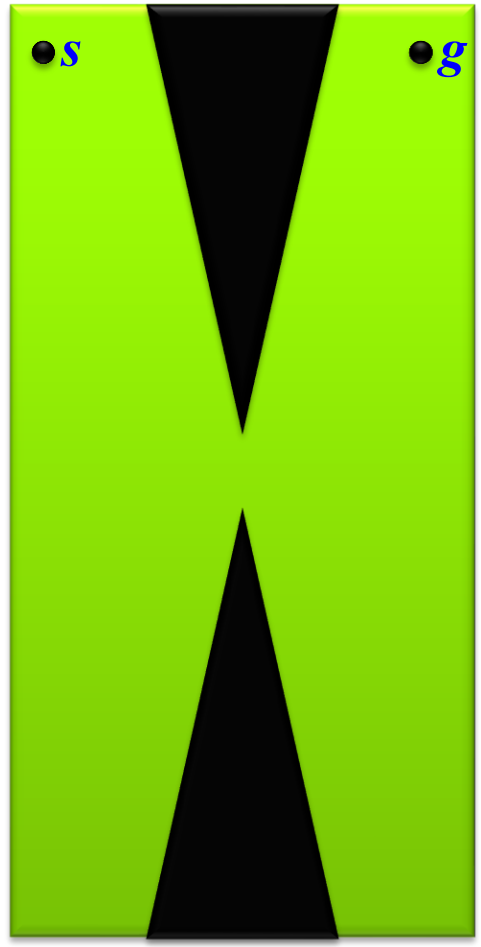
- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - Construction
      - **Expansion**

Phases of PRMs – Learning Phase

- Expansion Step – How to choose w(c)
  - One Method

    At the beginning of the expansion step, for every node c in N, compute $w(c)$ proportional to the failure ratio:

$$w(c) = \frac{r_f(c)}{\sum_{a \in N} r_f(a)}$$



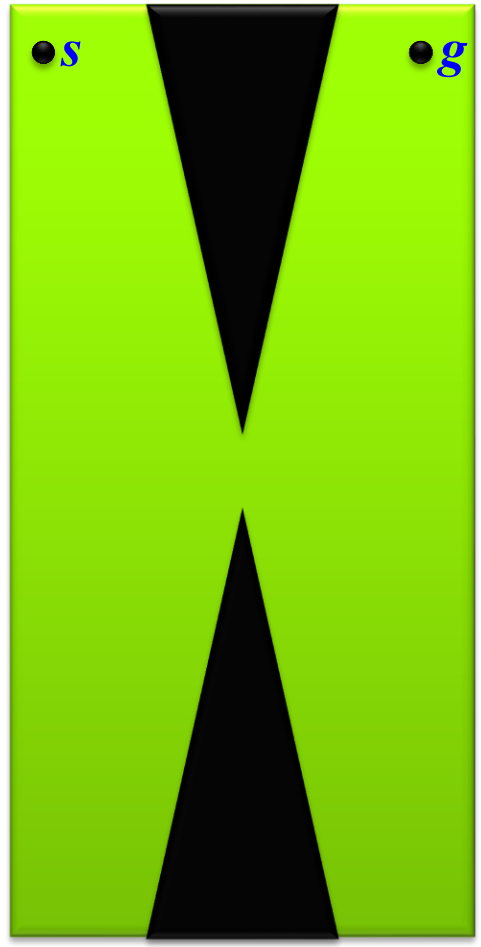
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - Construction
      - **Expansion**

Phases of PRMs – Learning Phase

- Expansion Step – How to choose w(c)
  - To expand a node *c*, compute a short random-bounce walk starting from *c*.
    - Repeatedly pick *random direction* of motion in *C-space* and move in this direction until obstacle is hit.
    - When *collision* occurs, choose a new random direction.
    - The final configuration *n* and the edge (*c, n*) are inserted into *R* and the path is memorized.
    - Connect *n* to the other connected components (like construction step)
    - Weights are only computed once at the beginning and not modified as nodes are added to *R*.



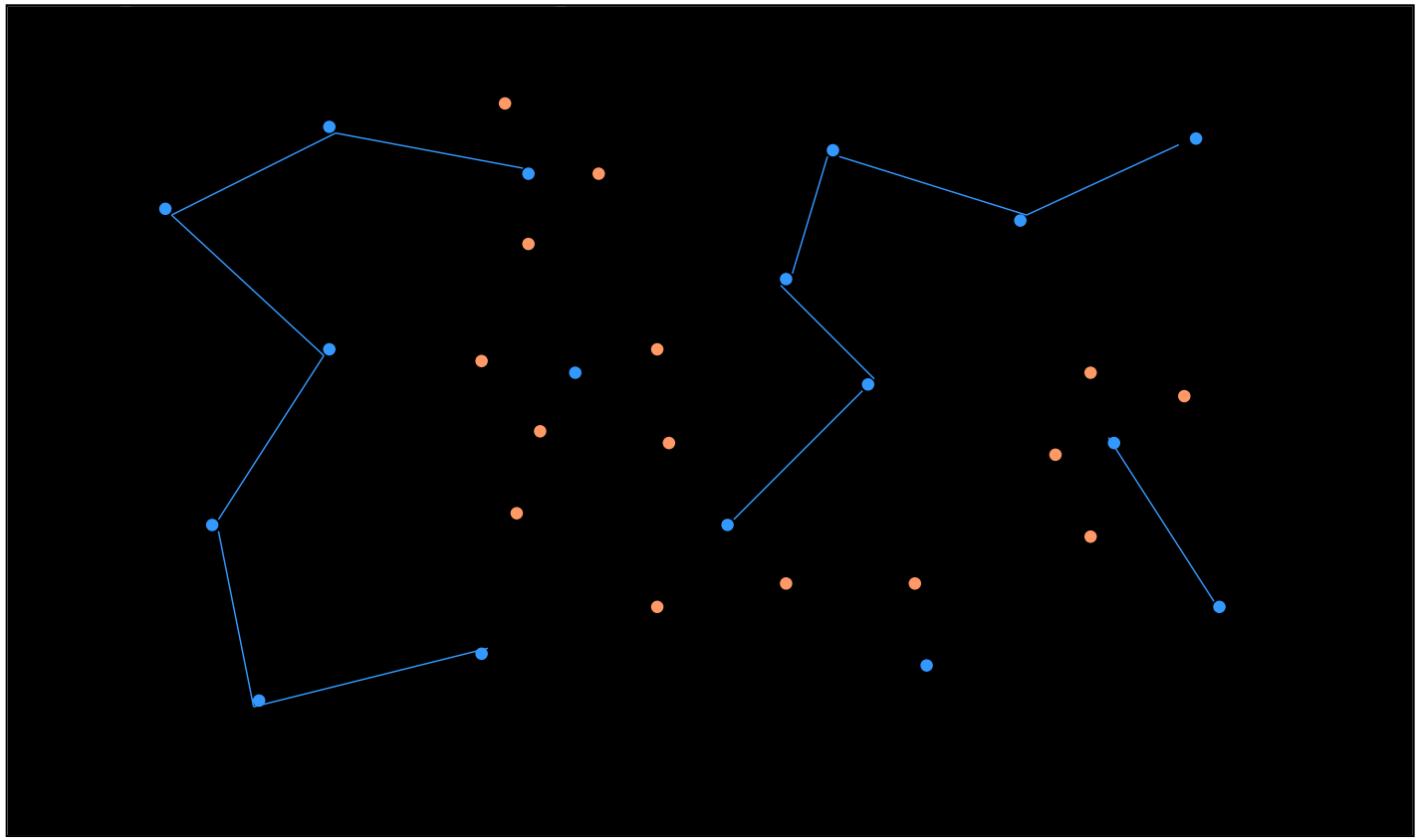
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - Construction
      - **Expansion**

Phases of PRMs – Learning Phase

- Expansion Step



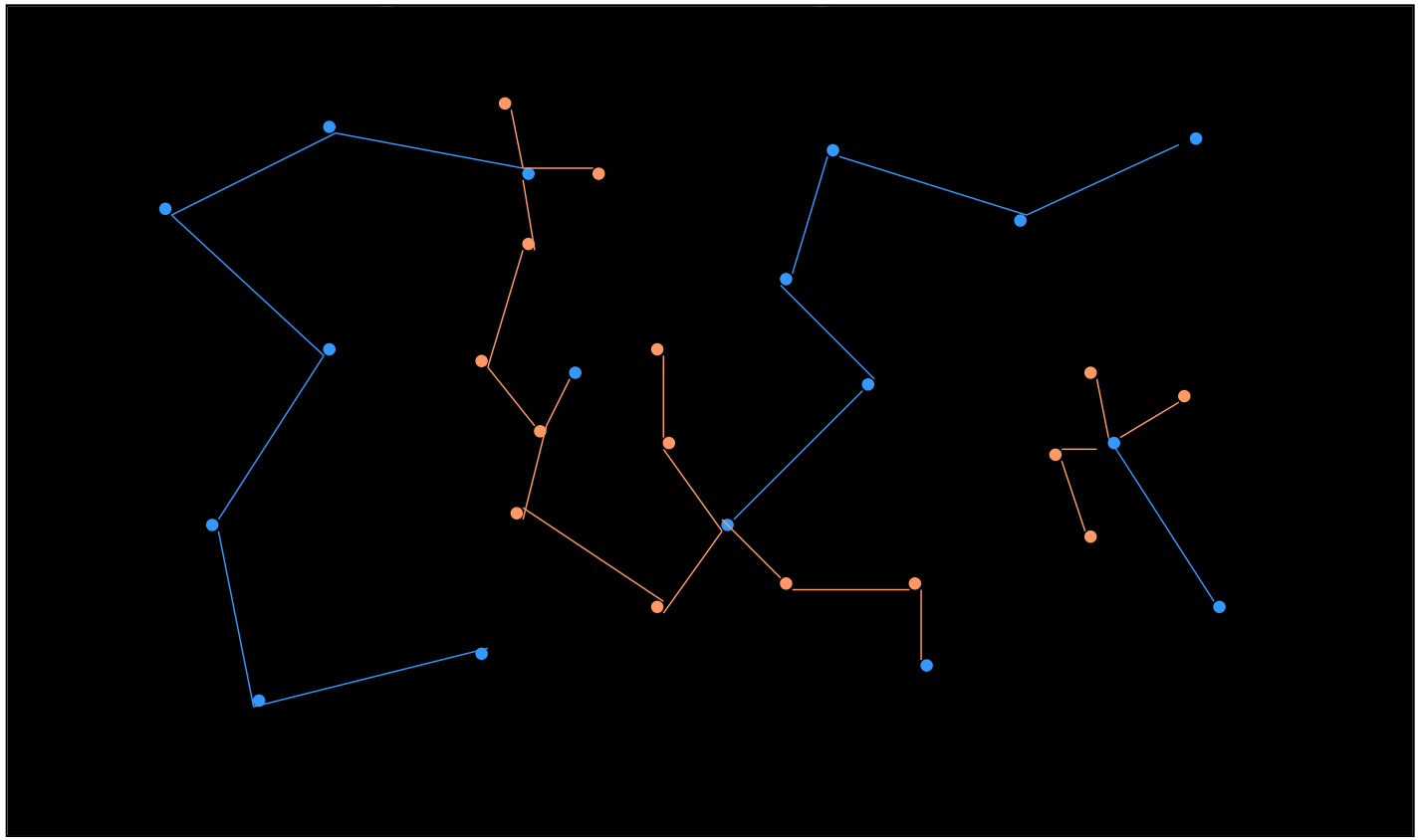
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - **Learning Phase**
      - Construction
      - **Expansion**

Phases of PRMs – Learning Phase

- Expansion Step



Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - + Learning Phase
    - **+ Query Phase**

Phases of PRMs – Query Phase

- Objective: To find a path between an arbitrary start and goal configuration, using the roadmap constructed in the learning phase.
  - Find a path from the start and goal configurations to two nodes of the roadmap
  - Search the graph to find a sequence of edges connecting those nodes in the roadmap
  - Concatenating the successive segments gives a feasible path for the robot

Sampling Based Methods

- Revision
- Sampling Methods
- PRM Approach
  - Phases
    - + Learning Phase
    - + Query Phase
      - Steps

Phases of PRMs – Query Phase

- Steps
  - Let start configuration be s
  - Let goal configuration be g
  - Try to connect s and g to Roadmap R at two nodes $\hat{s}$ and $\hat{g}$, with feasible paths P_s and P_g. If this fails, the query fails.
  - Compute a path P in R connecting $\hat{s}$ to $\hat{g}$.
  - Concatenate P_s , P and reversed P_g to get the final path.

Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - + Learning Phase
    - + **Query Phase**
      - Steps
      - **Finding Parameters**

Phases of PRMs – Query Phase

- How do we find P_s, P_g, and P?
  - Consider nodes in R in order of increasing distance from s (according to D) and try to connect s to each of them with the local planner, until one succeeds.
  - Random-bounce walks:
  - Try to connect to R with the local planner.
  - Re-compute paths found during the learning phase
  - Selective collision check.

Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - + Learning Phase
    - + **Query Phase**
      - Steps
      - Finding Parameters
      - **Final Step**

Phases of PRMs – Query Phase

- Final Step
  - If there are multiple connected components, sometimes the free C-space is not itself connected or the roadmap is not dense enough.
  - In this case, we try to connect s and g to nodes in the same component. If we can't do this, we end in failure. This is usually detected pretty quickly.

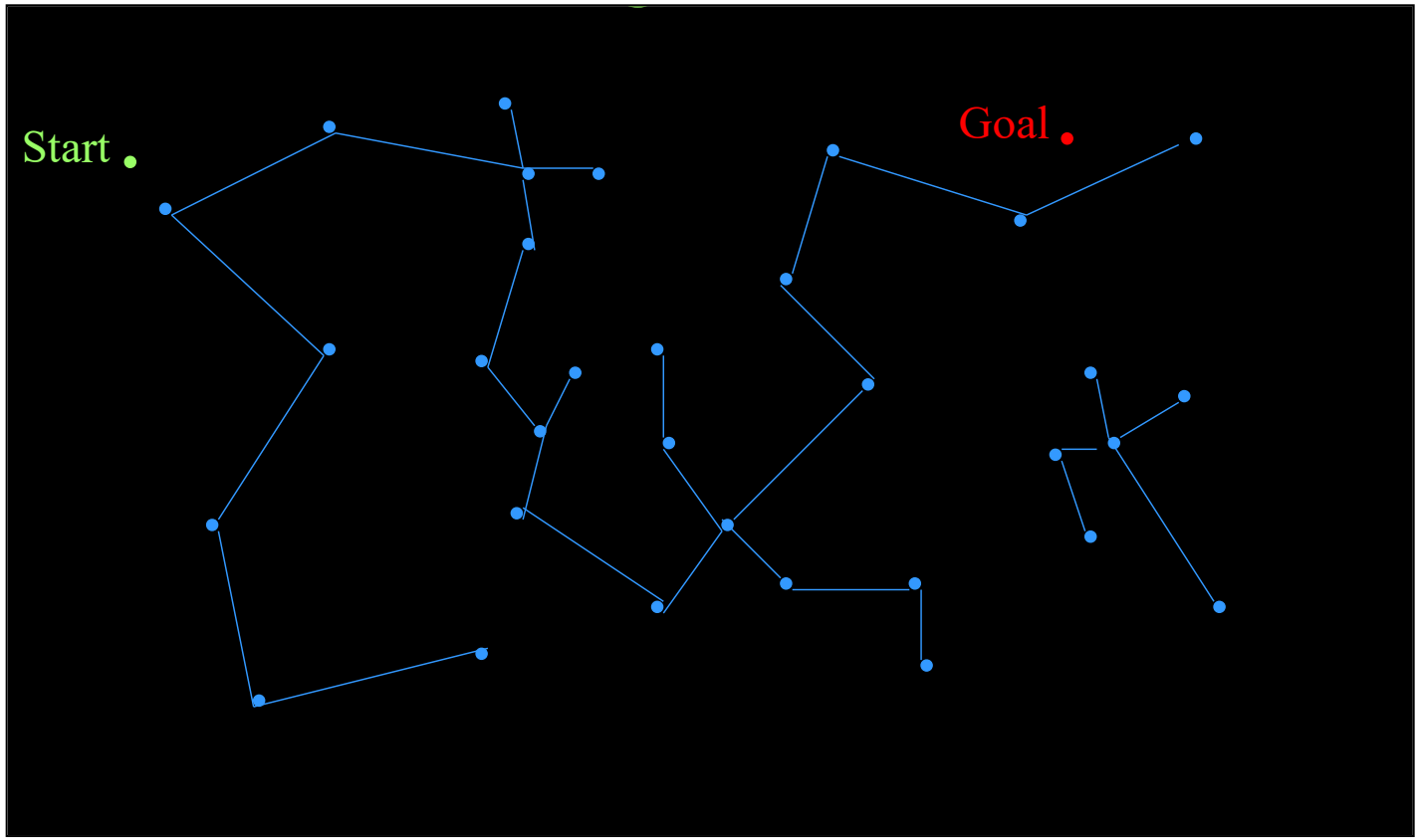
NATIONAL UNIVERSITY OF SCIENCES & TECHNOLOGY PAKISTAN
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - + Learning Phase
    - **+ Query Phase**
      - Steps
      - Finding Parameters
      - Final Step
      - **Example**

Phases of PRMs – Query Phase

- Select Start and Goal:



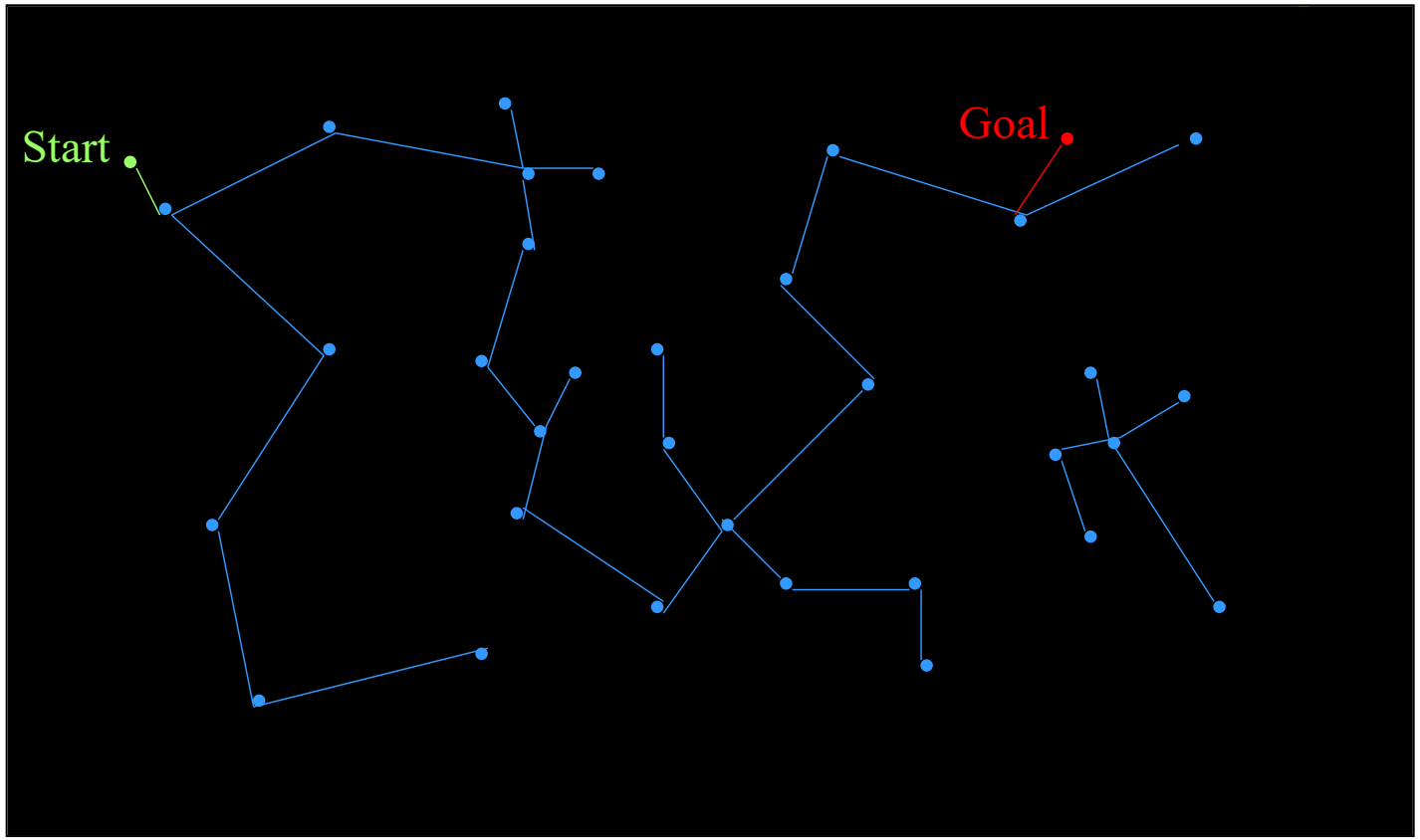
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - + Learning Phase
    - **+ Query Phase**
      - Steps
      - Finding Parameters
      - Final Step
      - **Example**

Phases of PRMs – Query Phase

- Connect Start and Goal to Road Map:



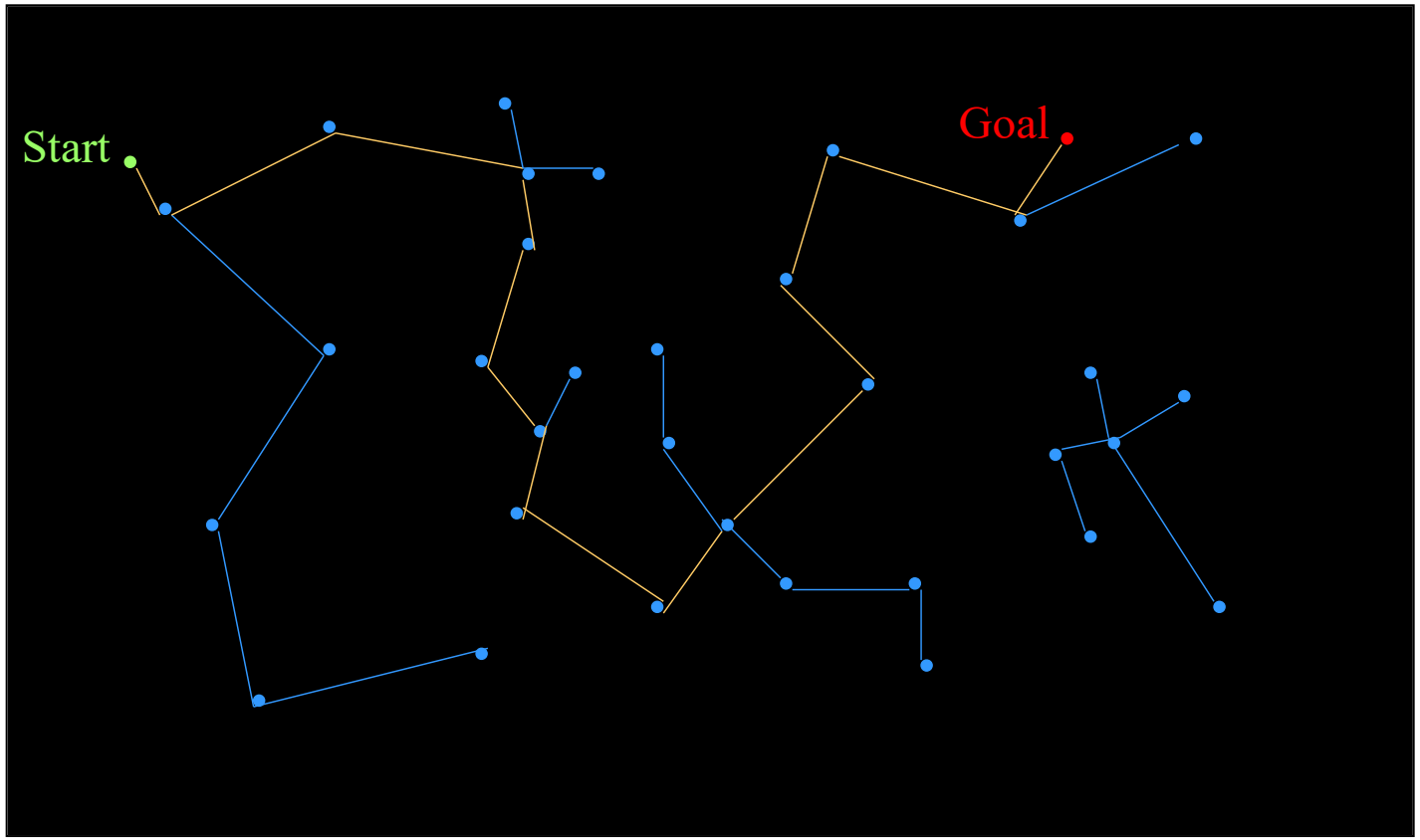
NATIONAL UNIVERSITY OF SCIENCES & TECHNOLOGY PAKISTAN
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - + Learning Phase
    - **+ Query Phase**
      - Steps
      - Finding Parameters
      - Final Step
      - **Example**

Phases of PRMs – Query Phase

- Find Path from Start to Goal:



Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - **Phases**
    - + Learning Phase
    - + **Query Phase**
      - Steps
      - Finding Parameters
      - Final Step
      - Example
      - **Failure?**

Phases of PRMs – Query Phase

- What if we Fail?
  - Maybe the roadmap was not adequate.
  - Could spend more time in the Learning Phase
  - Could do another Learning Phase and reuse R constructed in the first Learning Phase. In fact, Learning and Query Phases don't have to be executed sequentially.

Defining futures

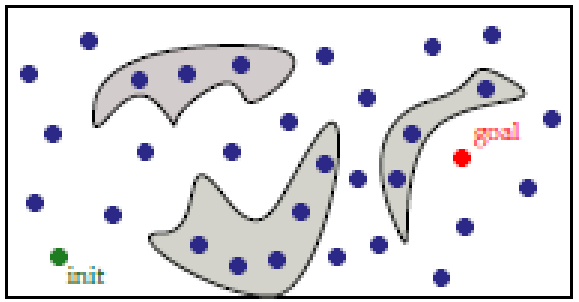
Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - Phases
  - **Applications**
    - **2D Point Robot**

PRM Applied to a 2D Point Robot

$q = (x, y) \leftarrow$ *SAMPLE* ( )

- $x \leftarrow \text{RAND}(min_x , max_x)$
- $y \leftarrow \text{RAND}(min_y , max_y)$



ISSAMPLECOLLISIONFREE (q)

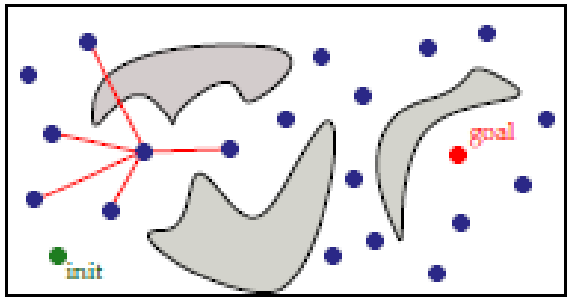
- Point inside/outside polygon test

PATH $\leftarrow$ *GENERATELOCALPATH* (q_a , q_b)

- Straight-line segment from point q_a to point q_b

ISPATHCOLLISIONFREE (*PATH*)

- Segment-polygon intersection test



Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - Phases
  - **Applications**
    - + 2D Point Robot
    - + **2D Rigid Body**

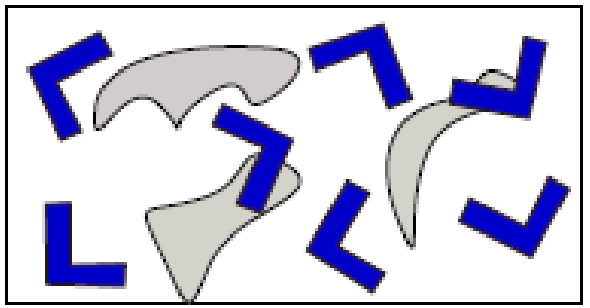
PRM Applied to a 2D Rigid Body

$q = (x, y, \theta) \leftarrow$ *SAMPLE* ( )

- $x \leftarrow$ RAND(min_x , max_x), $y \leftarrow$ RAND(min_y , max_y), $\theta \leftarrow$ RAND($-\pi$, π)

ISSAMPLECOLLISIONFREE (q)

- Place rigid body in position and orientation specified by q
- Polygon-polygon intersection test



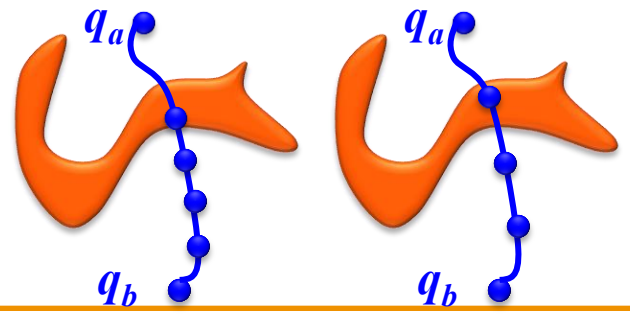
PATH $\leftarrow$ *GENERATELOCALPATH* (q_a , q_b)

- Continuous function parameterized by time: *PATH* : [0; 1] $\rightarrow Q$
- Starts at q_a and ends at q_b : *PATH* (0) = q_a , *PATH* (1) = q_b
- Many possible ways of defining it, e.g., by linear interpolation

$$PATH\ (t) = (1 - t) * q_a + t * q_b$$

ISPATHCOLLISIONFREE (*PATH*)

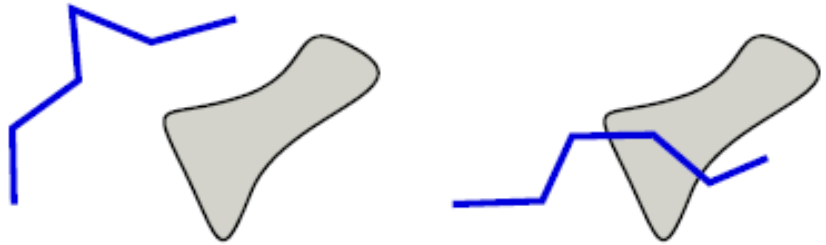
- Segment-polygon intersection test
- Subdivision Approach



Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - Phases
  - **Applications**
    - + 2D Point Robot
    - + 2D Rigid Body
    - + **Articulated Chain**

PRM Applied to an Articulated Chain



$q = (\theta_1, \theta_2, \ldots, \theta_n) \leftarrow$ *SAMPLE* ( )

- $\theta_i \leftarrow$ RAND $(-\pi, \pi)$, $\forall$

ISSAMPLECOLLISIONFREE (q)

- Place chain in configuration q (*forward kinematics*)
- Check for collisions with obstacles

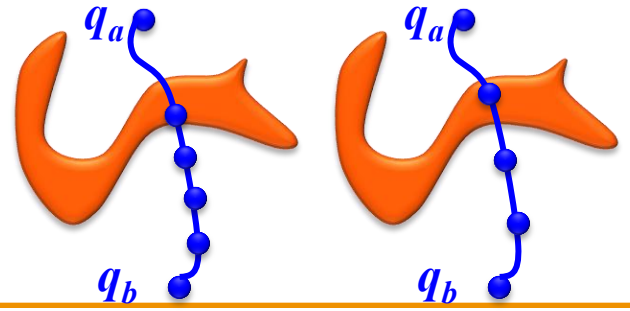
PATH $\leftarrow$ *GENERATELOCALPATH* (q_a, q_b)

- Continuous function parameterized by time: *PATH* : [0; 1] $\rightarrow Q$
- Starts at q_a and ends at q_b : *PATH* $(0) = q_a$, *PATH* $(1) = q_b$
- Many possible ways of defining it, e.g., by linear interpolation

$$PATH\ (t) = (1 - t) * q_a + t * q_b$$

ISPATHCOLLISIONFREE (*PATH*)

- Segment-polygon intersection test
- Subdivision Approach



Defining futures

Sampling Based Methods

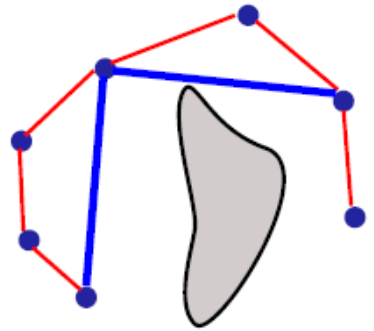
- Revision
- Sampling Methods
- PRM Approach
  - Phases
  - Applications
  - Post Processing

Path Smoothing

- Solution paths produced by PRM planners tend to be long and non-smooth (due to sampling and edge connections)
- Post processing is commonly used to improve the quality of the paths
- A common practice is to repeatedly replace long paths by short paths

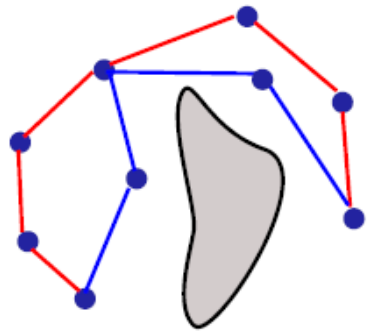
SMOOTHPATH $(q_1, q_2, \ldots, q_n)$ - One Version

1: for several times do
2: select i and j uniformly at random from $1, 2, \ldots, n$
3: attempt to directly connect q_i to q_j
4: If successful, remove the in-between nodes, i.e., $q_{i+1}, \ldots q_j$



SMOOTHPATH $(q_1, q_2, \ldots, q_n)$ – Another Version

1: for several times do
2: select i and j uniformly at random from $1, 2, \ldots, n$
3: $q \leftarrow$ generate collision-free sample
4: attempt to connect q_i to q_j through q
5: if successful, remove the in-between nodes, i.e., $q_{i+1}, \ldots q_j$



Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - Phases
  - Applications
  - Post Processing
  - **Connection Strategies**

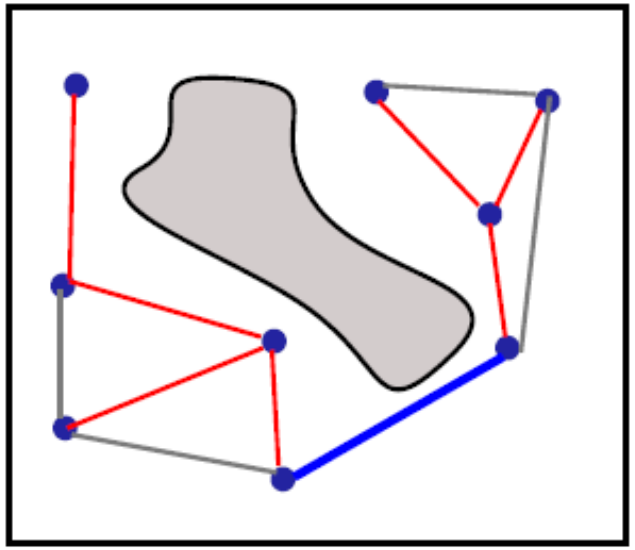
Roadmap with no Cycles

- Edge in cycle does not improve roadmap connectivity

- Edge is added to roadmap only if it connects two different roadmap components

```
1: if SAMEROADMAPCOMPONENT (q_a, q_b) = false then
2:     PATH ← GENERATEPATH (q_a, q_b)
3: if ISPATHCOLLISIONFREE (PATH) = true then
4:     (q_a, q_b) : PATH←PATH
5:     E ← E ∪ {(q_a, q_b)}
```

- Disjoint-set data structure is used to speed up computation of *SAMEROADMAPCOMPONENT* (q_a, q_b)



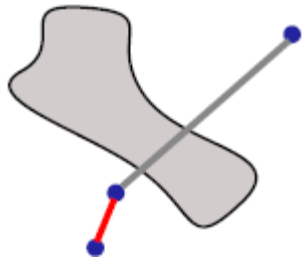
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - Phases
  - Applications
  - Post Processing
  - **Connection Strategies**

Connecting Roadmap Nodes to Nearest Neighbours

- *Edges between neighbouring nodes are more likely to be collision free than edges between far away nodes*



- Common practice is to attempt to connect each node to k of its nearest neighbours
- Nearest neighbours defined by distance metric $\rho : Q \times Q \rightarrow R^{\geq 0}$
  - geodesic distances, i.e., shortest-path distances according to topology
  - weighted combination of translation and rotation components
  - Euclidean distance between selected robot points

  Good metrics reflect likelihood of successful connections
- Numerous algorithms/data structures for nearest-neighbors computations, e.g., *kd-tree*, *R-tree*, *M-tree*, *V-tree*, *PR-tree*, *GNAT*, *iDistance*, *CoverTree*

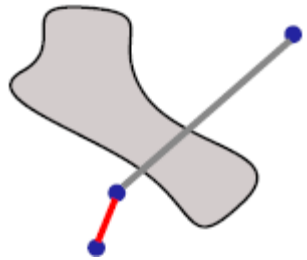
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - Phases
  - Applications
  - Post Processing
  - **Connection Strategies**

Connecting Roadmap Nodes to Nearest Neighbours

- *Edges between neighbouring nodes are more likely to be collision free than edges between far away nodes*



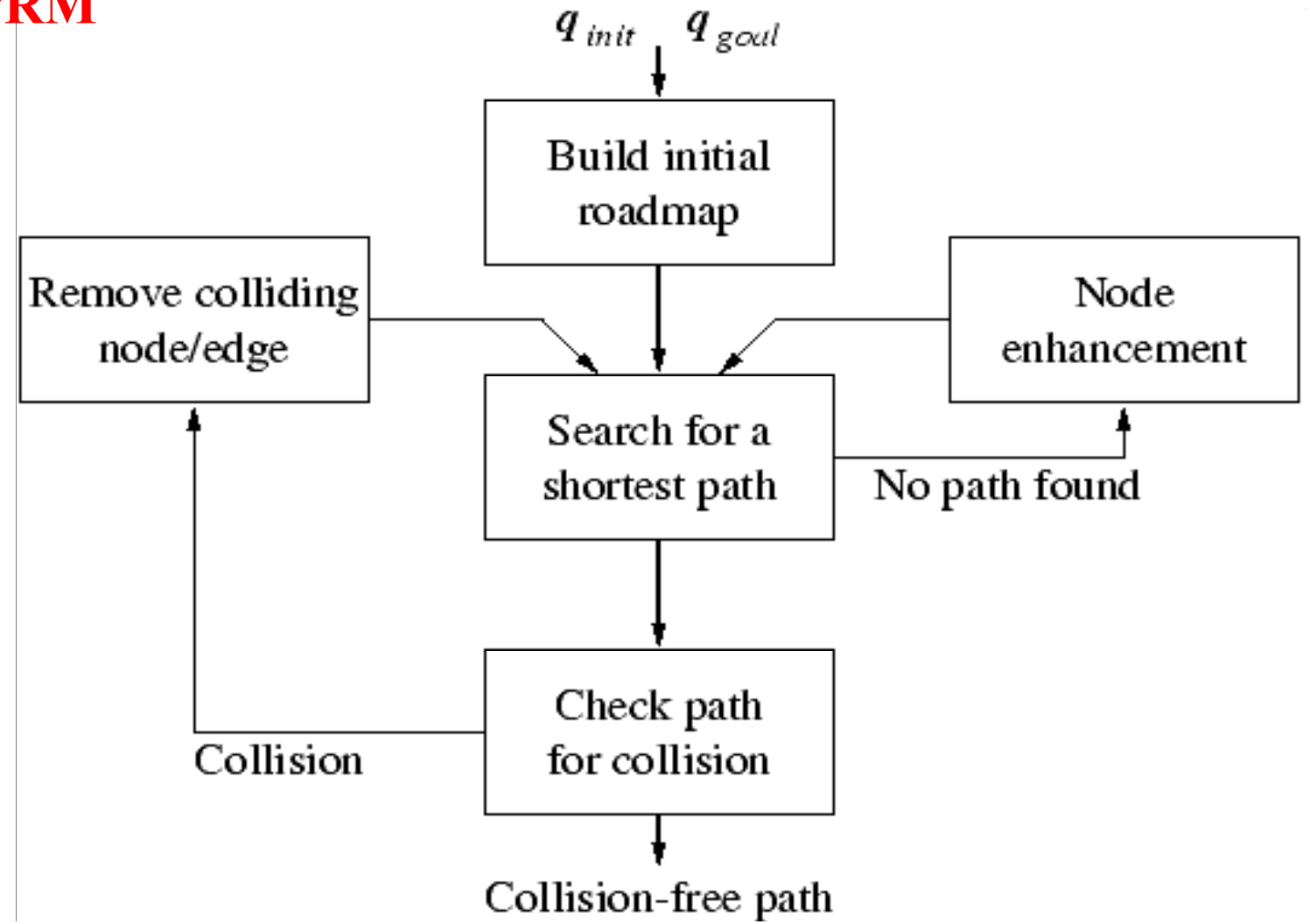
- Computational challenges of nearest neighbours in high-dimensional spaces
  - Efficiency deteriorates rapidly
  - Not much better than brute-force approach

- Alternative approach is to compute approximate nearest neighbours
  - Minimal losses in accuracy of neighbours
  - No loss in accuracy of overall path planner
  - Significant computational gains

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - Phases
  - Applications
  - Post Processing
  - **Connection Strategies**

Lazy PRM

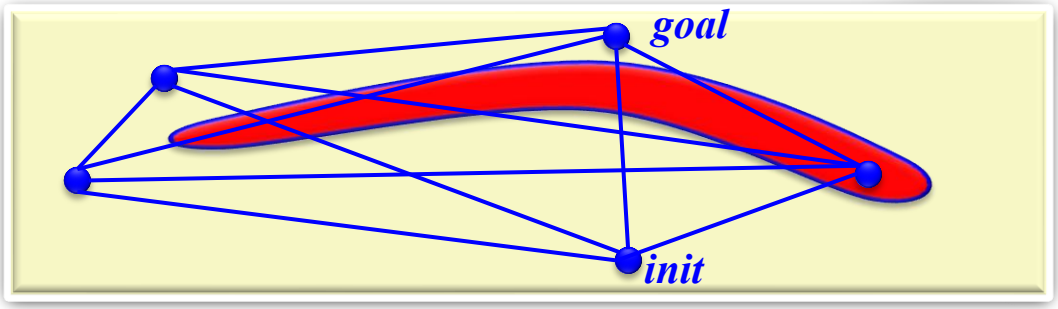


Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - Phases
  - Applications
  - Post Processing
  - **Connection Strategies**

Lazy PRM



Perform collision checking when necessary

LAZYROADMAPCOLLISIONCHECKING

1: V←V ∪ {q_{init}, q_{goal}} ; E ← 0;
2: for several times do
3: q ← generate config uniformly at random; q_i.*checked* ← *false*; V←V ∪ {q}
4: for each pair (qa; qb) 2 V V do
5: (q_a, q_b) .*res* ← 1.0; (q_a, q_b).*path* ← *GENERATEPATH* (q_a, q_b); E←E ∪ {(q_a, q_b)}

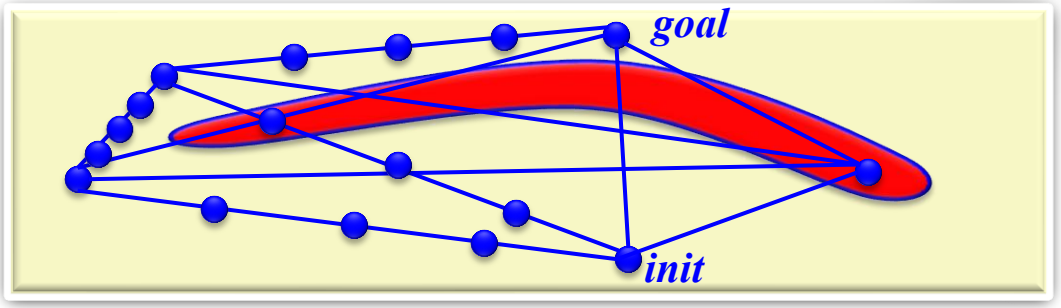
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - Phases
  - Applications
  - Post Processing
  - **Connection Strategies**

Lazy PRM

Perform collision checking when necessary



LAZYROADMAPCOLLISIONCHECKING

1: for several times do
2: $[q_1, q_2, \ldots, q_n] \leftarrow$ search $G = (V, E)$ for sequence of edges connecting q_{init} to q_{goal}
3: for $i = 1, 2, \ldots, n$ do
4: if $q_i.checked = false$ and *ISCONFIGCOLLISIONFREE* $(q_i) = false$ then
5: remove q_i from roadmap; goto line 2
6: else
7: $q_i.checked \leftarrow true$
8: while no edge collisions are found and minimum resolution not reached do
9: for $i = 1, 2, \ldots, n - 1$ do
10: $(q_i, q_{i+1}).res \leftarrow (q_i, q_{i+1}).res/2$; check $(q_i, q_{i+1}).path$ at resolution $(q_i, q_{i+1}).res$
11: if collision found in $(q_i, q_{i+1}).path$ then
12: remove (q_i, q_{i+1}) from roadmap; goto line 2
13: return $(q_i, q_{i+1}).\text{path} \circ \ldots \circ (q_{n-1}, q_n).path$

Defining futures

Sampling Based Methods

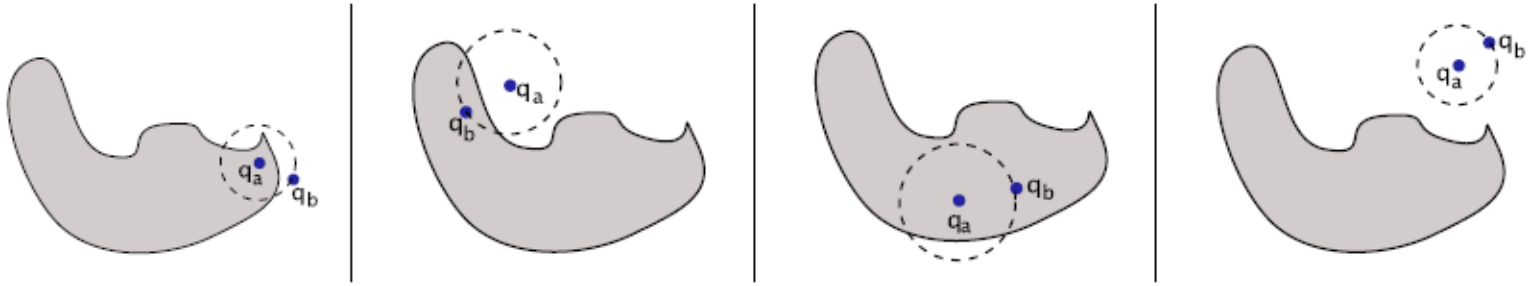
- Revision
- Sampling Methods
- PRM Approach
  - Phases
  - Applications
  - Post Processing
  - Connection Strategies

Gaussian Sampling in PRM

- *Objective*: Increase sampling inside/near narrow passages
- *Approach*: Sample from a Gaussian distribution biased near the obstacles

GENERATECOLLISIONFREECONFIG

1: $q_a \leftarrow$ generate config uniformly at random
2: $r \leftarrow$ generate distance from Gaussian distribution
3: $q_b \leftarrow$ generate config uniformly at random at distance r from q_a
4: $ok_a \leftarrow$ *ISCONFIGCOLLISIONFREE*(q_a)
5: $ok_b \leftarrow$ *ISCONFIGCOLLISIONFREE*(q_b)
6: if $ok_a = true$ and $ok_b = false$ then return q_a
7: if $ok_a = false$ and $ok_b = true$ then return q_b
8: return null



Defining futures

Sampling Based Methods

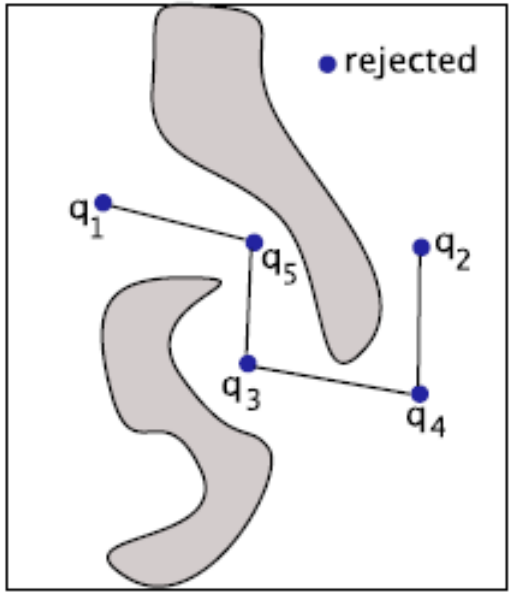
- Revision
- Sampling Methods
- PRM Approach
  - Phases
  - Applications
  - Post Processing
  - Connection Strategies

Visibility-Based Sampling in PRM

- *Objective*: Capture connectivity of config with few samples
- *Approach*: Generate samples that create new components or join existing components

GENERATECOLLISIONFREECONFIG

1: $q \leftarrow$ generate config uniformly at random
2: if *ISCONFIGCOLLISIONFREE*(q) = *true* then
3: if q belongs to a new roadmap component then
4: return q
5: if q connects two roadmap components then
6: return q
7: return null



q_1: creates new roadmap component
q_2: creates new roadmap component
q_3: creates new roadmap component
q_4: connects two roadmap components
q_5: connects two roadmap components

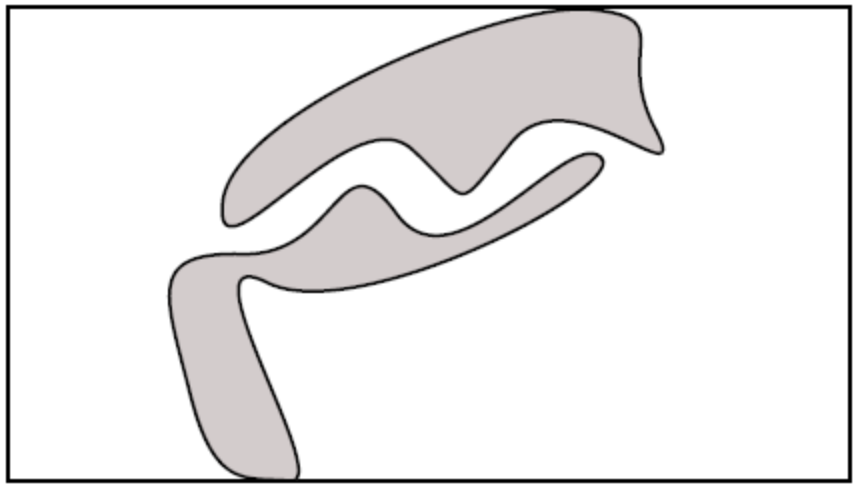
Defining futures

Sampling Based Methods

- Revision
- Sampling Methods
- **PRM Approach**
  - Phases
  - Applications
  - Post Processing
  - Connection Strategies
  - **Narrow Passage Problem**

Narrow-Passage Problem

- Probability of generating samples via uniform sampling in a narrow passage is low due to the small volume of the narrow passage
- Generating samples inside a narrow passage may be critical to the success of the path planner
- Objective is then to design sampling strategies that can increase the probability of generating samples inside narrow passages



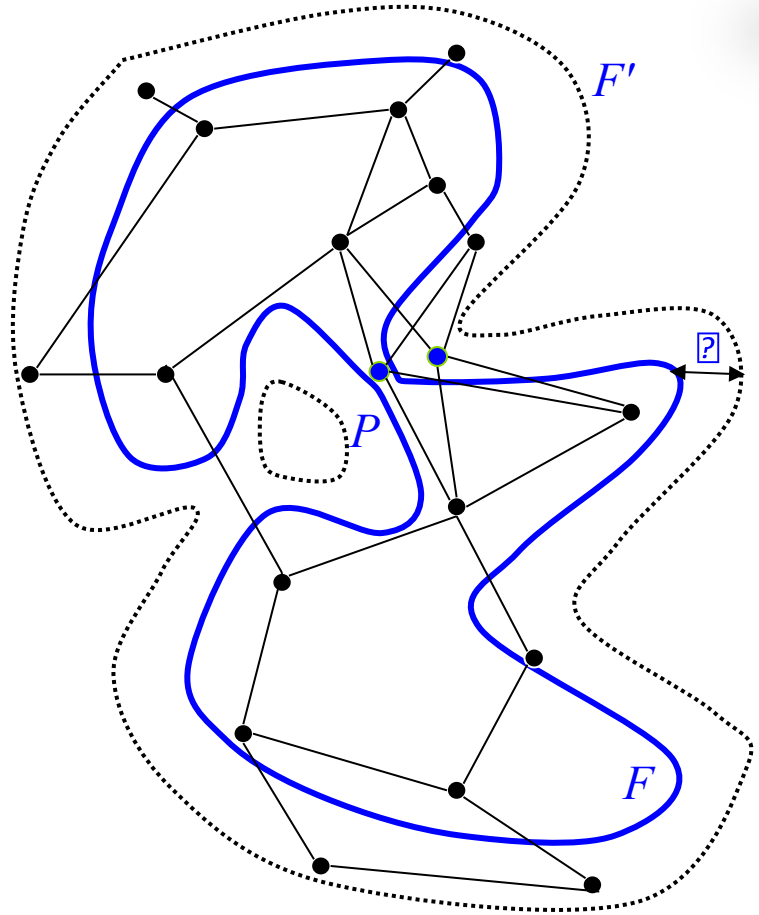
Defining futures

Sampling Based Methods

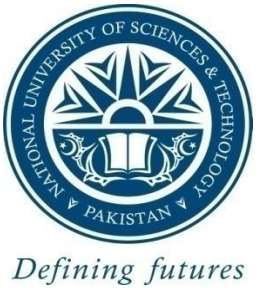
- Revision
- Sampling Methods
- **PRM Approach**
  - Phases
  - Applications
  - Post Processing
  - Connection Strategies
  - **Narrow Passage Problem**

Narrow-Passage Problem

- Dilate free space F to F' by allowing penetration distance ? into the obstacles
- Construct Roadmap R' in F'
- Push milestones and links that do not lie in F into F by performing local re-sampling operations. The outcome is a Roadmap R in F.
- Shrink the free space back to F
- Remove edges that are in collision with the obstacles



$F \rightarrow$ *Free Space*
$P \rightarrow$ *Narrow Space*
$F' \rightarrow$ *Dilated Space*



Defining futures

Sampling Based Methods

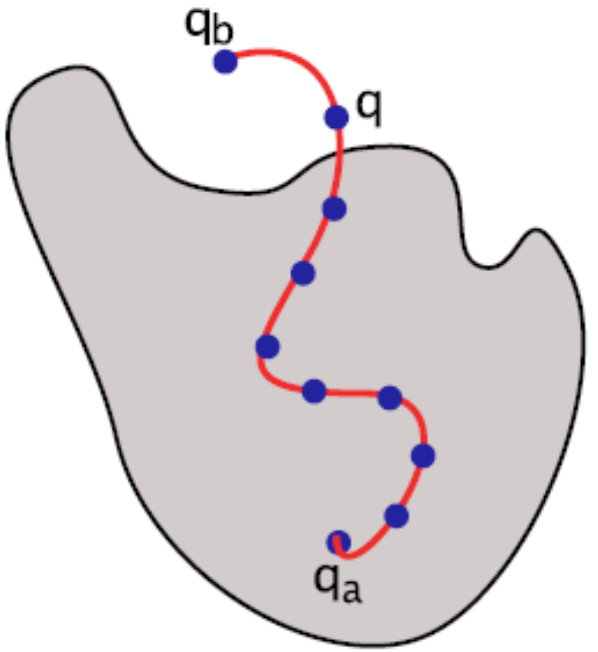
- Revision
- Sampling Methods
- **PRM Approach**
  - Phases
  - Applications
  - Post Processing
  - Connection Strategies
  - **Narrow Passage Problem**

Obstacle-Based Sampling in PRM

- *Objective*: Increase sampling Inside/near narrow passages
- *Approach*: Move samples in collision outside Obstacle boundary

GENERATECOLLISIONFREECONFIG

1: $q_a \leftarrow$ generate config uniformly at random
2: if *ISCONFIGCOLLISIONFREE*(q_a) = *true* then
3: return q_a
4: else
5: $q_b \leftarrow$ generate config uniformly at random
6: $path \leftarrow$ *GENERATEPATH* (q_a, q_b)
7: for $t = \delta$ to $|path|$ by δ do
8: if *ISCONFIGCOLLISIONFREE* ($path(t)$) then
9: return $path(t)$
10: return null



NATIONAL UNIVERSITY OF SCIENCES & TECHNOLOGY PAKISTAN
Defining futures

Sampling Based Methods

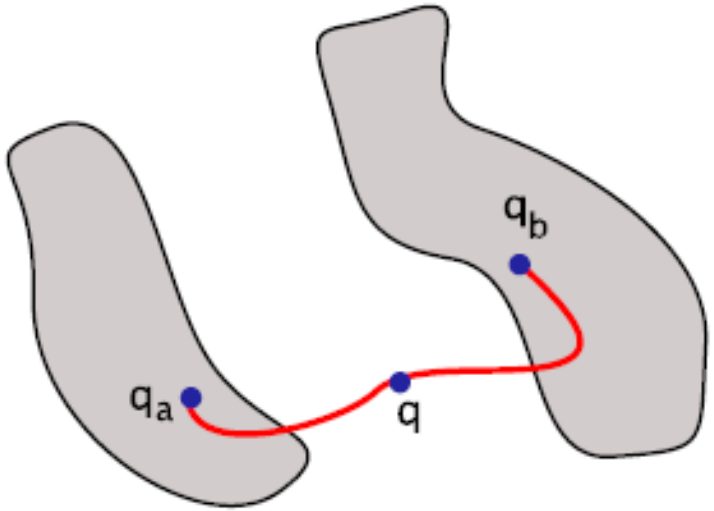
- Revision
- Sampling Methods
- **PRM Approach**
  - Phases
  - Applications
  - Post Processing
  - Connection Strategies
  - **Narrow Passage Problem**

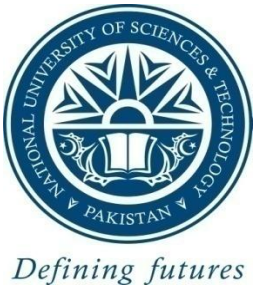
Bridge-Based Sampling in PRM

- *Objective*: Increase sampling Inside/near narrow passages
- *Approach*: Create *bridge* between samples in collision

GENERATECOLLISIONFREECONFIG

1: $q_a \leftarrow$ generate config uniformly at random
2: $q_b \leftarrow$ generate config uniformly at random
3: $ok_a \leftarrow$ *ISCONFIGCOLLISIONFREE*(q_a)
4: $ok_b \leftarrow$ *ISCONFIGCOLLISIONFREE*(q_b)
5: if $ok_a = false$ and $ok_b = false$ then
6: $path \leftarrow$ *GENERATEPATH* (q_a, q_b)
7: $q \leftarrow path\ (0.5|path|)$
8: if *ISCONFIGCOLLISIONFREE* (q) then
9: return q
10: return null





Defining futures